

QUIC Traffic Identification, Classification & Policing

Protocol Analysis and Study Document - Suvidh Jain

1. Introduction and Motivation

The internet's transport layer has been dominated by TCP for over four decades. TCP, paired with TLS for security and HTTP for application semantics, has served us remarkably well. But this stack has accumulated serious baggage that is becoming harder to ignore in 2026.

The Head-of-Line Blocking Problem: TCP guarantees in-order delivery of a byte stream. When HTTP/2 multiplexes multiple logical streams over a single TCP connection and a single packet is lost, TCP stalls the *entire connection* until that one packet is retransmitted and received. Every multiplexed stream - even those whose data arrived perfectly fine - sits waiting. This is TCP-level head-of-line (HOL) blocking, and HTTP/2 cannot escape it because it sits above TCP.

The Handshake Tax: A fresh HTTPS connection requires a TCP three-way handshake (1 RTT) followed by a TLS 1.3 handshake (1 RTT). That is a minimum of 2 round trips before a single byte of application data can flow. On a 200ms RTT link - common on mobile networks - that is 400ms of dead time. For short-lived connections (a quick API call, a DNS query), this overhead is often larger than the actual data transfer.

Ossification: TCP headers are sent in the clear. Over decades, middleboxes firewalls, NATs, load balancers, WAN optimizers - have learned to inspect and sometimes modify TCP headers. This has made it extraordinarily difficult to evolve TCP. Any new TCP option or behaviour change risks being dropped or mangled by some middlebox somewhere. The TCP protocol has, in effect, become ossified - frozen not by the spec but by the network infrastructure built around it.

The NAT Rebinding Problem: TCP connections are identified by the 4-tuple (source IP, source port, destination IP, destination port). When a mobile phone switches from Wi-Fi to cellular, its source IP changes, and every TCP connection dies. The application must reconnect, re-handshake, and restart. There is no mechanism in TCP for a connection to survive an address change.

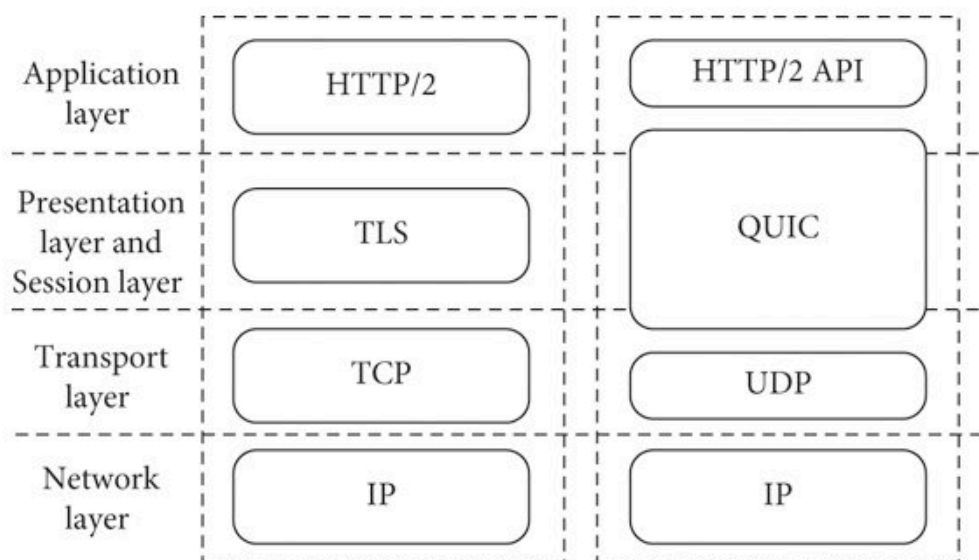
QUIC was designed from scratch to solve all of these problems simultaneously. Originally developed by Google (as gQUIC, deployed experimentally from 2013), QUIC was subsequently standardized by the IETF as RFC 9000 (published May 2021). It is a UDP-based, encrypted, multiplexed transport protocol that integrates transport and cryptographic handshakes into a single operation, eliminates HOL blocking through independent streams, and supports connection migration through connection IDs rather than address tuples.

The decision to run over UDP is not incidental but strategic. UDP is a thin, minimal protocol that middleboxes largely pass through without inspection. By encrypting nearly everything above the UDP header, QUIC deliberately prevents middlebox interference, ensuring the protocol can evolve without ossification. But this encryption is precisely what creates challenges for network operators who need to apply QoS policies, rate limiting, and traffic classification which is the core problem this project addresses.

As of 2026, QUIC carries a significant and growing fraction of internet traffic. Google Chrome uses QUIC for all Google services, Facebook/Meta uses it extensively, and HTTP/3 (RFC 9114) runs exclusively over QUIC.

2. The Protocol Stack — Where QUIC Sits

To understand QUIC properly, we need to see exactly where it sits relative to the traditional stack it replaces.



A few things to notice here:

UDP is just a substrate: QUIC uses UDP purely for its demultiplexing capability (port numbers) and its ability to traverse NATs and middleboxes. QUIC does not rely on UDP for any transport semantics whatsoever, no reliability, no ordering, no flow control from UDP. All of that is built into QUIC itself. In a sense, UDP is just a "carrier wave" that gets QUIC packets through the network infrastructure without interference.

TLS is not a separate layer in QUIC: In the traditional stack, TLS sits between TCP and HTTP as a distinct protocol layer with its own record format and handshake messages. In QUIC, TLS 1.3's cryptographic handshake is embedded directly into QUIC's transport handshake. TLS messages are carried inside QUIC CRYPTO frames, and QUIC handles its own packet protection rather than using TLS records. This integration is what enables the 1-RTT (and 0-RTT) handshake.

QUIC is a userspace protocol: TCP is implemented in the OS kernel. QUIC is implemented in the application or in userspace libraries. This means QUIC can be updated independently of the operating system. A browser update can ship a new QUIC implementation. This is a deliberate design choice to prevent the kind of ossification that has plagued TCP.

3. QUIC Packet Format and Wire-Level Anatomy

QUIC defines two fundamental header forms: **Long Header** and **Short Header**.

3.1 The Common Invariant: The First Byte

RFC 8999 (Version-Independent Properties of QUIC) specifies that across all QUIC versions, present and future, the very first bit of every QUIC packet determines the header form:

```
Bit 0 = 1 → Long Header
Bit 0 = 0 → Short Header
```

This single bit is the most fundamental invariant of the protocol and is guaranteed to remain stable across all future QUIC versions. Any QUIC detector can rely on this.

3.2 Long Header Format

Long headers are used during connection establishment and for certain special packet types. Their structure (RFC 9000, Section 17.2):

```
Long Header Packet {
  Header Form (1 bit)           = 1
  Fixed Bit (1 bit)             = 1
  Long Packet Type (2 bits)
  Type-Specific Bits (4 bits)
  Version (32 bits)
  Dest Connection ID Len (8 bits)
  Dest Connection ID (0..160 bits)
  Src Connection ID Len (8 bits)
  Src Connection ID (0..160 bits)
  Type-Specific Payload (..)
}
```

Long Packet Types (2-bit field):

Value	Type	When Used
0x00	Initial	First handshake packets; carries ClientHello
0x01	0-RTT	Early application data before handshake completes
0x02	Handshake	Carries handshake completion messages
0x03	Retry	Server asks client to prove reachability

What is visible to a network observer in a Long Header packet:

- Header form bit (it is a Long Header)
- QUIC version (32-bit field, in the clear)
- Destination Connection ID and its length
- Source Connection ID and its length
- The packet type (Initial, 0-RTT, Handshake, Retry)

What is NOT visible:

- Packet number (encrypted via header protection)
- Payload content (encrypted)
- The type-specific bits in the first byte (partially protected)

The **Version** field is critical for identification. QUIC v1 uses version `0x00000001`. QUIC v2 (RFC 9369) uses `0x6b3343cf`. Version `0x00000000` is reserved for Version Negotiation packets. The version field being in the clear is an intentional design choice defined in RFC 8999.

3.3 Short Header Format

Once the handshake is complete and both sides have established 1-RTT keys, QUIC switches to Short Header packets for all subsequent communication:

```
Short Header Packet {
  Header Form (1 bit)           = 0
  Fixed Bit (1 bit)             = 1
  Spin Bit (1 bit)
  Reserved Bits (2 bits)
  Key Phase (1 bit)
  Packet Number Length (2 bits)
  Dest Connection ID (variable)
  Packet Number (8..32 bits)
  Payload (..)
}
```

What is visible to a network observer in a Short Header packet:

- Header form bit (it is a Short Header)
- The Fixed Bit (always 1, useful for detection)
- Spin Bit (used for passive RTT measurement; discussed in detail later)
- Destination Connection ID (but the observer must already know its length from observing the handshake, since the length is NOT encoded in the Short Header)

What is NOT visible:

- Source Connection ID (not present in Short Header at all, saves bytes)
- Packet number (encrypted)
- Key phase bit (protected)
- Reserved bits (protected)
- Payload (encrypted)

This is a critical observation: **Short Header packets do not contain a version field, do not contain a Source Connection ID, and do not encode the Destination Connection ID length.** If a network observer did not see the handshake, it cannot reliably parse Short Header packets because it does not know where the Connection ID ends and the encrypted payload begins.

Long Header

Header Form	Fixed Bit	Long Packet Type	Type Specific bits	Version ID	DCID Len	DCID	SCID Len	SCID
1 bit	1 bit	2 bits	4 bits	32 bits	8 bits	0-160 bits	8 bits	0-160 bits

Short Header

Header Form	Fixed Bit	Spin bits	Reserved	Key Phase	P	DCID	Packet Number	Protected payload
1 bit	1 bit	1 bit	2bits	1 bit	2 bits	160 bits	P+8 bits	

3.4 Packet Coalescing

An important subtlety: QUIC allows **multiple QUIC packets to be coalesced into a single UDP datagram**. For example, during the handshake, a server might send an Initial packet and a Handshake packet in the same UDP datagram. The receiver processes them sequentially.

This means a single UDP datagram does not necessarily contain a single QUIC packet. Any parser or detector must be aware of this possibility. The length field present in Long Header packets enables delineation of coalesced packets.

3.5 Frames within Packets

Each QUIC packet's decrypted payload contains one or more **frames**. Frames are the actual units of information exchange. RFC 9000 Section 19 defines the frame types:

Frame Type	Purpose
PADDING (0x00)	Padding to increase packet size (e.g., for PMTUD or to meet minimum size)
PING (0x01)	Keep-alive, soliciting an ACK
ACK (0x02-0x03)	Acknowledging received packets
RESET_STREAM (0x04)	Abruptly terminate a stream
STOP_SENDING (0x05)	Request that a peer stop sending on a stream
CRYPTO (0x06)	Carry TLS handshake messages
NEW_TOKEN (0x07)	Provide tokens for future 0-RTT
STREAM (0x08-0x0f)	Carry application data
MAX_DATA (0x10)	Connection-level flow control
MAX_STREAM_DATA (0x11)	Stream-level flow control
MAX_STREAMS (0x12-0x13)	Limit on stream creation

Frame Type	Purpose
DATA_BLOCKED (0x14)	Indicate flow control blocking
STREAM_DATA_BLOCKED (0x15)	Stream-level blocking signal
STREAMS_BLOCKED (0x16-0x17)	Stream limit blocking signal
NEW_CONNECTION_ID (0x18)	Provide new connection IDs for migration
RETIRE_CONNECTION_ID (0x19)	Retire old connection IDs
PATH_CHALLENGE (0x1a)	Validate a new path
PATH_RESPONSE (0x1b)	Respond to path validation
CONNECTION_CLOSE (0x1c-0x1d)	Close the connection
HANDSHAKE_DONE (0x1e)	Server confirms handshake completion

Since all frame content is encrypted, a network observer cannot see any of these. This is fundamentally different from TCP, where flags (SYN, ACK, FIN, RST) are visible in the clear header.

3.6 Decrypting QUIC Initial Packets

Initial packet keys are derivable by anyone who observes the Destination Connection ID. Here is the exact process, derived from RFC 9001, Section 5.2 and Appendix A.

Step 1: Extract the DCID from the Initial packet.

The DCID starts at byte offset 6 (after Header Form/Fixed/Type/Reserved bits [1 byte] + Version [4 bytes] + DCID Length [1 byte]). The DCID Length byte tells you how many bytes to read.

Step 2: Derive the Initial Secret.

```
initial_salt (QUIC v1) = 0x38762cf7f55934b34d179ae6a4c80cadccbb7f0a
initial_salt (QUIC v2) = 0x0dede3def700a6db819381be6e269dcbf9bd2ed9
```

```
initial_secret = HKDF-Extract(salt=initial_salt, IKM=client_dst_connection_id)
```

The salt differs between QUIC v1 and v2. It ensures that a v1 and v2 connection starting with the same DCID produce different keys, preventing cross-version confusion.

Step 3: Derive client and server Initial secrets.

```

client_initial_secret = HKDF-Expand-Label(
    secret=initial_secret,
    label="client in",    // note: this is the actual label string
    context="",          // empty
    length=32            // for SHA-256
)

server_initial_secret = HKDF-Expand-Label(
    secret=initial_secret,
    label="server in",
    context="",
    length=32
)

```

Step 4: Derive the specific keys from each secret.

For each direction (client/server), derive three values:

```

key = HKDF-Expand-Label(secret, "quic key", "", key_length)    // 16 bytes for AES-128
iv  = HKDF-Expand-Label(secret, "quic iv", "", 12)             // 12 bytes always
hp  = HKDF-Expand-Label(secret, "quic hp", "", key_length)     // header protection key

```

For QUIC v1 Initial packets, the cipher suite is always AES-128-GCM with SHA-256 (this is hardcoded, not negotiated). For QUIC v2, the Initial cipher suite is AES-128-GCM as well but the HKDF labels change to "quicv2 key", "quicv2 iv", "quicv2 hp".

Step 5: Remove Header Protection.

This must happen before decryption because you need the packet number to construct the AEAD nonce.

1. Let `pn_offset` = byte offset where the packet number begins
(after DCID, SCID, Token Length, Token, Length fields)
2. Sample 16 bytes starting at `(pn_offset + 4)` from the encrypted payload
3. Compute `mask = AES-ECB(hp_key, sample)`
// This produces 16 bytes; use the first 5
4. Unmask the first byte:
`first_byte ^= (mask[0] & 0x0f)` // only lower 4 bits for Long Header
// The lower 2 bits of the first byte now reveal packet number length
5. `pn_length = (first_byte & 0x03) + 1` // 1 to 4 bytes
6. Unmask the packet number bytes:
for `i` in `range(pn_length)`:
`packet_number_bytes[i] ^= mask[1 + i]`

Step 6: Construct the AEAD nonce and decrypt.

```

nonce = iv XOR (packet_number padded to 12 bytes, left-padded with zeros)

associated_data = everything from the first byte through the packet number (now unprotected)

plaintext = AES-128-GCM-Decrypt(
    key=key,
    nonce=nonce,
    aad=associated_data,
    ciphertext=remaining_payload    // includes 16-byte authentication tag
)

```

Step 7: Parse the decrypted payload.

The plaintext contains QUIC frames. Look for CRYPTO frames (type 0x06). The CRYPTO frame payload contains raw TLS handshake messages. Parse the TLS ClientHello to extract:

SNI (Server Name Indication): In the server_name extension (extension type 0x0000)

ALPN (Application-Layer Protocol Negotiation): In extension type 0x0010. Values like "h3" indicate HTTP/3, "doq" indicates DNS-over-QUIC.

Supported Versions: In the supported_versions TLS extension (type 0x002b)

QUIC Transport Parameters: In extension type 0x0039 (for QUIC v1) or 0x0057 (for client Initial in compatible version negotiation). These contain the endpoint's advertised configuration.

What Transport Parameters reveal (RFC 9000, Section 18.2):

max_idle_timeout	How long the connection will stay alive without activity
max_udp_payload_size	Maximum UDP datagram the endpoint is willing to receive
initial_max_data	Initial connection-level flow control limit
initial_max_stream_data_bidi_local/remote	Initial stream-level flow control limits
initial_max_streams_bidi/uni	How many concurrent streams the endpoint allows
active_connection_id_limit	Maximum number of CIDs the endpoint will store
max_datagram_frame_size	Whether QUIC Datagrams (RFC 9221) are supported and their max size

These parameters provide valuable metadata for classification: a video streaming application might advertise very different values than an API client.

Performance consideration: This entire decryption process involves HKDF operations, AES-ECB (for header protection removal), and AES-128-GCM (for payload decryption). On a modern CPU with AES-NI instructions, this takes roughly 1-5 microseconds per packet. At 10 Gbps line rate, you may be processing hundreds of thousands of Initial packets per second during peak hours. Whether this is feasible depends on your deployment scale and architecture.

4. The QUIC Handshake

QUIC merges the transport handshake (analogous to TCP's SYN/SYN-ACK/ACK) with the cryptographic handshake (analogous to TLS ClientHello/ServerHello/Finished) into a single process that completes in **1 RTT** for a fresh connection and **0 RTT** for a resumed connection.

4.1 Encryption Levels

QUIC operates at four distinct **encryption levels**, each with its own set of keys:

Encryption Level	Keys Derived From	Used For
Initial	Derived from the Destination Connection ID using a well-known salt (published in RFC 9001 Section 5.2). These keys provide NO confidentiality from a determined observer. They are derivable by anyone who sees the packet.	Initial handshake packets (ClientHello, ServerHello)
Handshake	Derived from the TLS handshake (server's handshake traffic secret)	Handshake completion messages
0-RTT (Early Data)	Derived from a previously established PSK (Pre-Shared Key) from a prior session	Early application data sent before handshake completes
1-RTT (Application)	Derived from the completed TLS 1.3 handshake (application traffic secrets)	All post-handshake application data

The progression through these levels is what defines the handshake flow.

4.2 Step-by-Step 1-RTT Handshake

Step 1: Client sends Initial packet

The client constructs a QUIC Initial packet containing:

- **Long Header** with packet type `Initial` (0x00)
- **Version** set to `0x00000001` (QUIC v1)
- **Destination Connection ID (DCID)**: A randomly generated value chosen by the client. The DCID in this first packet is used to derive Initial encryption keys (RFC 9001, Section 5.2). Both client and server independently compute the same Initial keys from this DCID.
- **Source Connection ID (SCID)**: Another randomly generated value chosen by the client. The server will use this as the DCID in its response.
- **CRYPTO frame** containing the TLS 1.3 **ClientHello** message, which includes: supported cipher suites, supported QUIC versions, TLS extensions (including SNI, ALPN, key share for the key exchange, supported groups, etc.)
- **PADDING frames** to pad the packet to at least **1200 bytes**. This minimum size requirement (RFC 9000, Section 14.1) is a deliberate design choice: it ensures that the path MTU is at least 1200 bytes before proceeding, and it prevents amplification attacks (an attacker cannot trigger a large server response with a tiny spoofed packet).

The Initial keys are derived as follows (RFC 9001, Section 5.2):

```
initial_salt = 0x38762cf7f55934b34d179ae6a4c80cadccbb7f0a (for QUIC v1)
initial_secret = HKDF-Extract(initial_salt, client_dst_connection_id)
client_initial_secret = HKDF-Expand-Label(initial_secret, "client in", "", Hash.length)
server_initial_secret = HKDF-Expand-Label(initial_secret, "server in", "", Hash.length)
```

Since the DCID is sent in the clear and the salt is published in the RFC, **anyone can derive the Initial keys and decrypt Initial packets**. Initial encryption exists not for confidentiality but for consistency. All QUIC packets use the same protection mechanism, and this prevents middleboxes from treating Initial packets differently (which would lead to ossification).

Step 2: Server sends Initial + Handshake packets (often coalesced in one UDP datagram)

The server processes the ClientHello and responds with:

a) An Initial packet containing:

- A CRYPTO frame with the TLS 1.3 **ServerHello** message (which includes the server's key share, selected cipher suite, selected version)
- An ACK frame acknowledging the client's Initial packet
- The server chooses its own Source Connection ID (SCID)

b) One or more Handshake packets containing:

- CRYPTO frames with the remaining TLS 1.3 handshake messages: **EncryptedExtensions, Certificate, CertificateVerify, Finished**
- These are encrypted with Handshake keys, which are derived from the TLS handshake. An observer **cannot** decrypt these.

At this point, the server has computed both Handshake keys and 1-RTT (Application) keys. The server can actually start sending 1-RTT data immediately after its Finished message if it chooses, though it will not have confirmation that the client has received the Handshake messages yet.

Step 3: Client completes the handshake

The client:

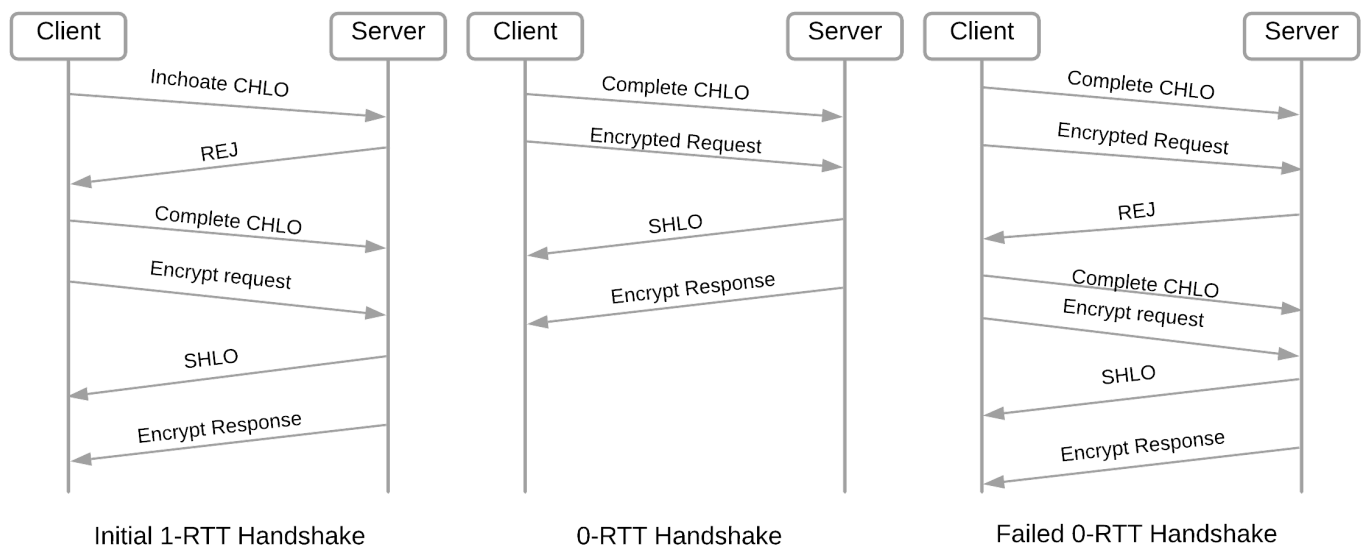
- Decrypts the server's Initial packet using the Initial keys
- Processes the ServerHello to compute Handshake keys
- Decrypts the Handshake packets
- Validates the server's certificate and CertificateVerify
- Computes 1-RTT (Application) keys
- Sends a Handshake packet containing its own TLS **Finished** message
- Immediately begins sending 1-RTT application data

At this point, **1 RTT has elapsed**, and the client is already sending encrypted application data. Compare this to TCP+TLS, which would require 2 RTTs minimum (3 RTTs with TCP Fast Open disabled).

Step 4: Server confirms handshake completion

After receiving the client's Finished message, the server sends a **HANDSHAKE_DONE** frame in a 1-RTT packet. This frame is unique to QUIC (it has no TLS analogue) and tells the client that the server has confirmed the

handshake is complete. After this, both sides discard their Initial and Handshake keys.



4.3 The Retry Mechanism

There is an important variant: the server may choose to send a **Retry packet** instead of its Initial packet. This happens when the server wants to:

1. **Validate the client's address** (prevent IP spoofing in amplification attacks)
2. **Choose a different Connection ID**

The Retry packet is not encrypted (it uses a special Retry Integrity Tag for tamper-proof verification). It contains:

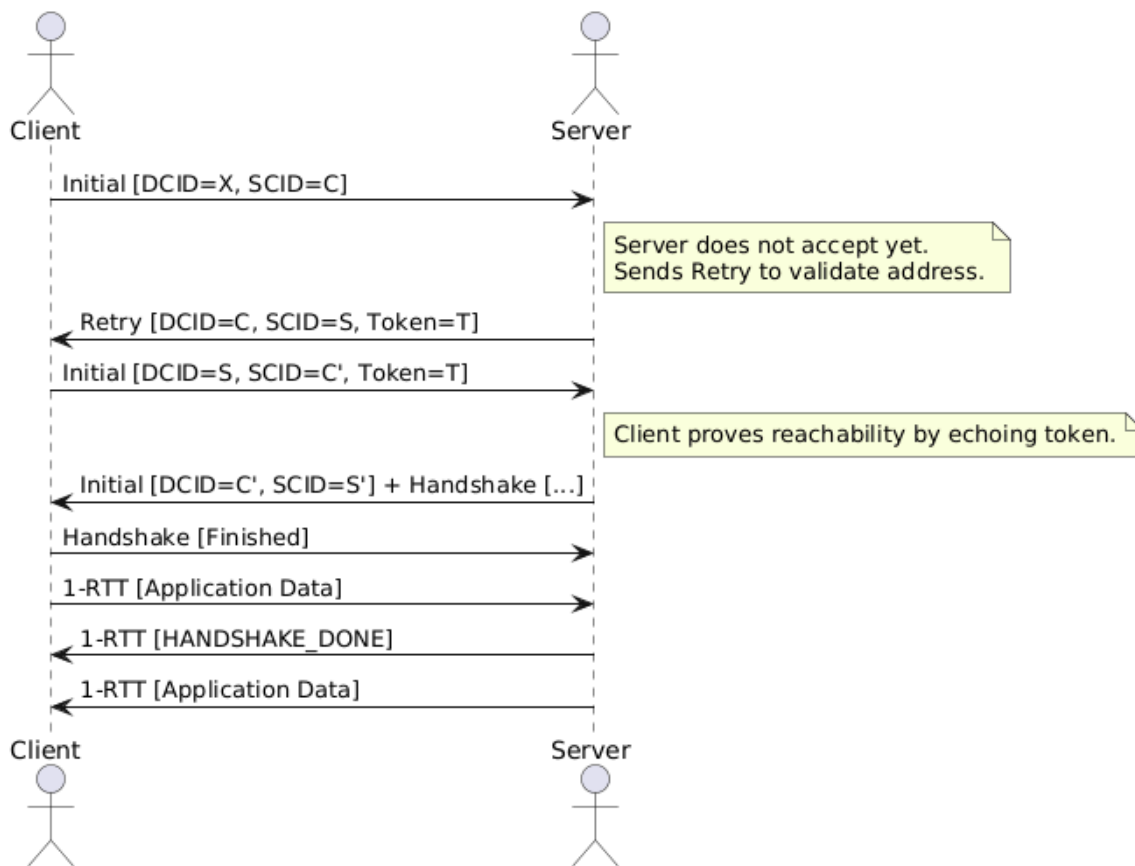
- A new Source Connection ID chosen by the server
- A **Retry Token** i.e. an opaque blob that the client must echo back

Upon receiving a Retry, the client must:

- Send a new Initial packet with the new DCID (from the Retry's SCID)
- Include the Retry Token in the new Initial packet
- Use the new DCID to derive new Initial keys

This effectively adds 1 RTT to the handshake but provides address validation. The Retry mechanism is important for DoS mitigation.

QUIC Retry Handshake Flow

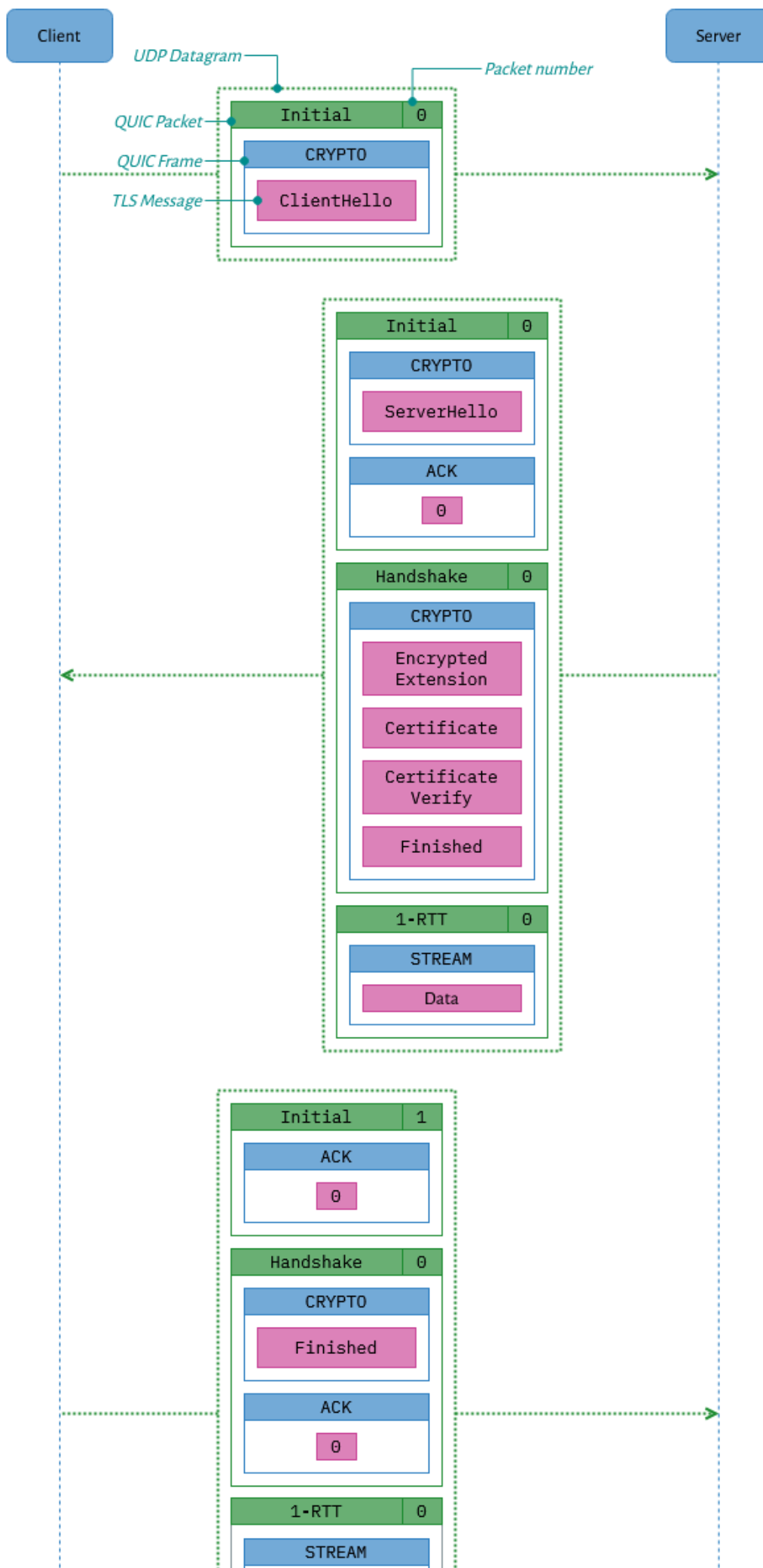


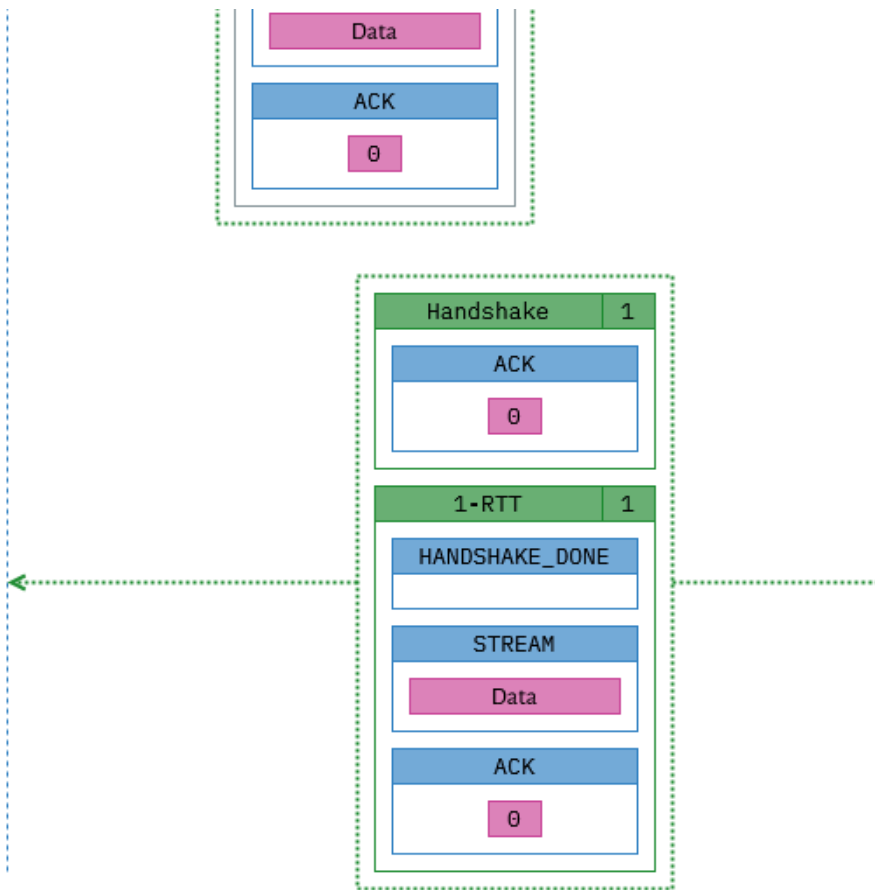
4.4 Handshake on the Wire : What an Observer Sees

This is directly relevant to flow detection:

Handshake Phase	What Is Visible	What Is Hidden
Client Initial	QUIC version, DCID, SCID, packet type, packet size (~1200 bytes minimum)	ClientHello content (encrypted with derivable keys), SNI (encrypted in payload, but derivable from Initial keys)
Server Initial	QUIC version, DCID, SCID, packet type	ServerHello content
Server Handshake	DCID, SCID, packet type	Certificate, EncryptedExtensions, Finished
Client Handshake	DCID, SCID, packet type	Client Finished
1-RTT Data	DCID (only), Spin Bit	Everything else

An important observation: because Initial keys are derivable, a determined observer could decrypt the ClientHello and extract the SNI (Server Name Indication), revealing which website the client is connecting to. However, the **Encrypted Client Hello (ECH)** extension (currently in draft) aims to encrypt the SNI as well, which would close this visibility window.





5. Connection IDs

Connection IDs are one of QUIC's most distinctive and operationally significant features.

5.1 Purpose and Design Rationale

TCP identifies a connection by the 4-tuple: (source IP, source port, destination IP, destination port). This works as long as neither endpoint changes its network address. The moment a mobile device moves from Wi-Fi to cellular, the source IP changes, and the TCP connection is irrecoverably broken.

QUIC decouples connection identity from network addresses by using **Connection IDs** i.e. opaque byte sequences embedded in the QUIC header. Each endpoint independently chooses the Connection IDs that the *peer* will use when sending packets to it. This allows an endpoint to change its IP address or port while maintaining the same logical QUIC connection.

5.2 How Connection IDs Work

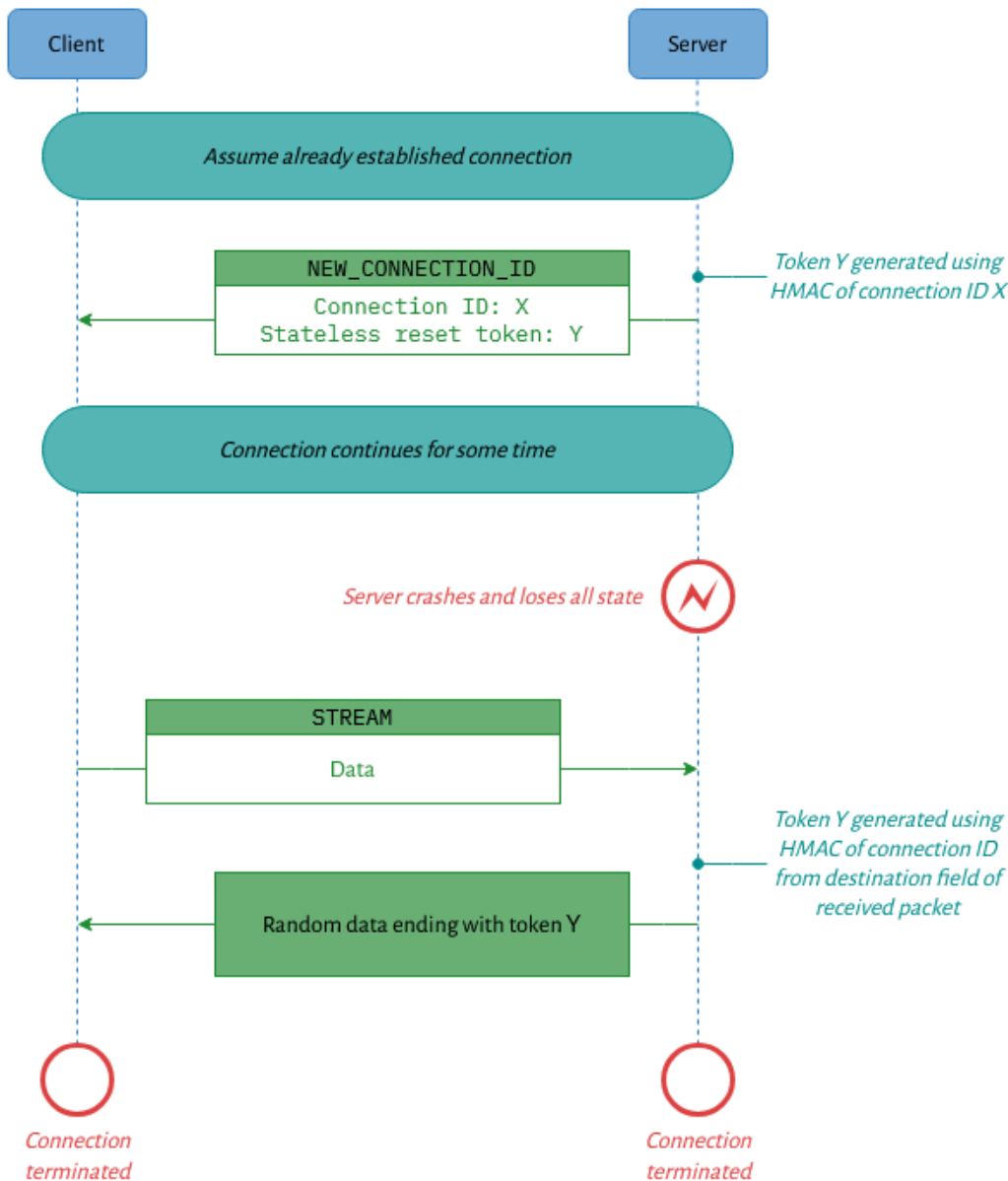
Each endpoint maintains a **set of Connection IDs** that the peer may use. At any given time, the peer sends packets with one of these Connection IDs in the Destination Connection ID field. The receiving endpoint uses the Connection ID to look up the connection state, rather than relying on the source address.

Key properties of Connection IDs (RFC 9000, Section 5.1):

- **Length:** 0 to 20 bytes. Yes, a zero-length Connection ID is permitted (in which case the 4-tuple must be used for demultiplexing, sacrificing migration capability).

- **Opacity:** Connection IDs are opaque to the peer. The *issuing* endpoint may embed structure in them (e.g., routing information for load balancers), but the *peer* treats them as opaque values.
- **Multiple active CIDs:** Each endpoint can have multiple valid Connection IDs simultaneously. This is essential for connection migration (discussed in Section 9).
- **Unidirectional issuance:** Each endpoint issues CIDs that the *other side* uses to address packets to it. A server issues CIDs that the client puts in the DCID field when sending to the server, and vice versa.

5.3 Connection ID Lifecycle



1. Initial CIDs (During Handshake):

- The client generates a random DCID for the first Initial packet (used to derive Initial keys).
- The client also generates its own SCID.
- The server, in its first Initial response, provides its chosen SCID (which becomes the client's new DCID for subsequent packets).
- After the Retry (if any), both sides converge on their respective CIDs.

2. NEW_CONNECTION_ID Frames (Post-Handshake):

After the handshake, either endpoint can issue new Connection IDs using the `NEW_CONNECTION_ID` frame (type

0x18). This frame contains:

- A **sequence number** (monotonically increasing)
- A **Retire Prior To** field (tells the peer to retire all CIDs with sequence numbers below this value)
- The **Connection ID** itself
- A **Stateless Reset Token** (16 bytes). It used for stateless connection resets

3. RETIRE_CONNECTION_ID Frames:

When an endpoint is done with a Connection ID, it sends a `RETIRE_CONNECTION_ID` frame (type 0x19) indicating the sequence number of the CID being retired. The issuing endpoint can then reuse the resources associated with that CID.

4. Connection ID Rotation:

Endpoints are expected to rotate Connection IDs periodically, especially when migrating to a new network path. The reason is privacy: if a client moves from Wi-Fi to cellular and continues using the same Connection ID, an observer who sees traffic on both networks can link the two and track the user. By switching to a new Connection ID when migrating, the client makes it difficult for network observers to correlate the old and new paths.

5.4 Implications for Flow Tracking

1. **Connection IDs change over time.** A long-lived QUIC connection may use many different Connection IDs. A flow tracker that indexes by Connection ID must handle CID changes.
2. **Connection IDs differ per direction.** The CID in packets from client→server is different from the CID in packets from server→client. They are not the same connection identifier. They are independently chosen by each endpoint.
3. **Short Header packets only contain the DCID.** There is no SCID. So after the handshake, you only see one CID per packet, and it is the CID chosen by the *receiving* endpoint.
4. **The CID length is not encoded in Short Header packets.** If you did not observe the handshake, you do not know how many bytes of the Short Header constitute the Connection ID. This is a deliberate anti-ossification measure.
5. **Zero-length CIDs are valid.** If both endpoints use zero-length CIDs, the QUIC flow can only be identified by the traditional 4-tuple, and the flow becomes indistinguishable in structure from some other UDP-based protocols.
6. **Load balancer considerations:** RFC 9312 acknowledges that load balancers often need to route packets to the correct server based on the Connection ID. Servers behind load balancers typically encode routing information in the CID in a way the load balancer understands. This is specified in a separate document (draft-ietf-quic-load-balancers).

5.5 Stateless Reset

When a server crashes and restarts, it has lost all connection state. If a client continues sending packets with a Connection ID the server no longer recognizes, the server needs a way to signal "I don't know this connection" without having any state.

QUIC handles this through **Stateless Reset** (RFC 9000, Section 10.3). The server sends a packet that looks like an ordinary Short Header packet but ends with a **Stateless Reset Token** : a 16-byte value that was previously communicated to the client in the `NEW_CONNECTION_ID` frame. The client recognizes the token and tears down the connection.

The important detail: a Stateless Reset is designed to be **indistinguishable from a regular Short Header packet** to a network observer. It prevents middleboxes from detecting and interfering with connection teardown.

5.6 Connection Termination

QUIC connections can end in three ways and the network observer cannot directly see any of them.

1. **Idle Timeout:** Both endpoints negotiate an idle timeout via the `max_idle_timeout` transport parameter during the handshake. If no packets are sent or received for this duration, the connection silently closes. Neither side sends any closing packet, the connection simply expires. The effective idle timeout is the minimum of the two endpoints' advertised values. Typical idle timeouts range from 30 seconds to 10 minutes. Google's QUIC implementation defaults to 30 seconds.
2. **Immediate Close (CONNECTION_CLOSE frame):** Either endpoint can send a `CONNECTION_CLOSE` frame (type `0x1c` for QUIC errors, `0x1d` for application errors) to immediately terminate the connection. This frame is sent in a 1-RTT packet and is therefore encrypted and invisible to the network.
-> After sending `CONNECTION_CLOSE`, the endpoint enters a "closing period" during which it responds to any incoming packets with another `CONNECTION_CLOSE` (to handle the case where the original was lost). After the closing period (typically $3 \times \text{PTO}$), the endpoint discards all connection state.
3. **Stateless Reset:** As described in Section 5.5, a Stateless Reset looks like an ordinary Short Header packet and is indistinguishable from normal traffic to an observer.

Implications for flow tracking:

Since none of these termination mechanisms produce a visible signal, a flow tracker must use timeout-based expiry. If no packets matching a flow's identifiers (Connection ID, 5-tuple) are observed for a configurable duration, the flow is considered terminated. The choice of timeout involves a trade-off:

- **Too short:** Active but idle connections are prematurely expired, causing state to be lost. If the connection resumes activity, it appears as a "new" flow, fragmenting accounting.
- **Too long:** Resources (memory, state table entries) are held for connections that have already closed, potentially leading to state exhaustion under heavy load.

A reasonable starting point is 60-120 seconds, which covers the common idle timeout values while avoiding excessive state retention. For more precise tracking, decrypting the Initial packet to extract the `max_idle_timeout` transport parameter can allow per-flow timeout tuning.

6. Version Negotiation — Mechanism and Invariants

6.1 The Version Negotiation Mechanism

QUIC supports multiple protocol versions, and the Version Negotiation mechanism allows client and server to agree on a version.

How it works:

1. The client sends an Initial packet with its preferred version in the Version field (e.g., `0x00000001` for QUIC v1).
2. If the server does not support that version, it responds with a **Version Negotiation packet**.

3. The Version Negotiation packet has a special structure:

- Header Form = 1 (Long Header)
- Version = `0x00000000` (this specific value signals "this is a Version Negotiation packet")
- It echoes the client's DCID and SCID
- The payload contains a list of versions the server supports

4. The client picks a mutually supported version and restarts the handshake.

```
Version Negotiation Packet {
  Header Form (1 bit)      = 1
  Unused (7 bits)
  Version (32 bits)        = 0x00000000
  DCID Len (8 bits)
  DCID (..)
  SCID Len (8 bits)
  SCID (..)
  Supported Version 1 (32 bits)
  Supported Version 2 (32 bits)
  ...
}
```

The Version Negotiation packet as described above is **not authenticated**. An on-path attacker could forge a Version Negotiation packet, downgrading the connection to an older, potentially weaker version. This is known as a **version downgrade attack**.

6.2 Compatible Version Negotiation (RFC 9368)

To address the downgrade vulnerability, RFC 9368 introduces **Compatible Version Negotiation**. The key idea is:

- Versions can declare themselves "compatible" with other versions. Compatible versions share the same handshake mechanism and can transition between each other *during* the handshake.
- The client includes a `version_information` transport parameter in its ClientHello, listing all versions it supports.
- The server includes its supported versions in its transport parameters.
- Since transport parameters are part of the TLS handshake (and therefore authenticated), a downgrade attack would cause the handshake to fail.

This mechanism ensures that version negotiation is tamper-proof while maintaining the ability to evolve the protocol.

6.3 QUIC Version Invariants (RFC 8999)

RFC 8999 formally specifies the properties that are constant across **all** QUIC versions, past and future. The invariants are:

1. **The first bit** of every QUIC packet indicates the header form (1=Long, 0=Short).
2. **Long Header packets** always contain: a Version field at bytes 1-4, a DCID Length and DCID, and a SCID Length and SCID, in that order.
3. **The Version field** in Long Header packets is at a fixed offset (bytes 1-4).
4. **Version `0x00000000`** always indicates a Version Negotiation packet.
5. **Short Header packets** always begin with a 0 bit.

Everything else may change between versions. The packet type encoding, frame types, encryption algorithms, header protection mechanisms — all of these are version-specific. A QUIC detector that relies on anything beyond these invariants risks breaking when a new QUIC version is deployed.

6.4 Greasing

To prevent ossification (where middleboxes start depending on specific version numbers), QUIC uses a concept called **GREASE (Generate Random Extensions And Sustain Extensibility)**. Reserved version numbers of the form `0x?a?a?a` (where `?` is any hex digit) must be tolerated by all implementations. A server that receives an Initial packet with a greased version must not treat it as an error. It simply does not support that version and should send a Version Negotiation packet. This ensures that the version negotiation path is always exercised.

7. Stream Multiplexing and Flow Control

7.1 Streams

Streams are the primary mechanism for structured data exchange in QUIC. A stream is an ordered sequence of bytes, similar to a TCP connection, but QUIC supports multiple concurrent streams within a single connection.

Stream types (RFC 9000, Section 2.1):

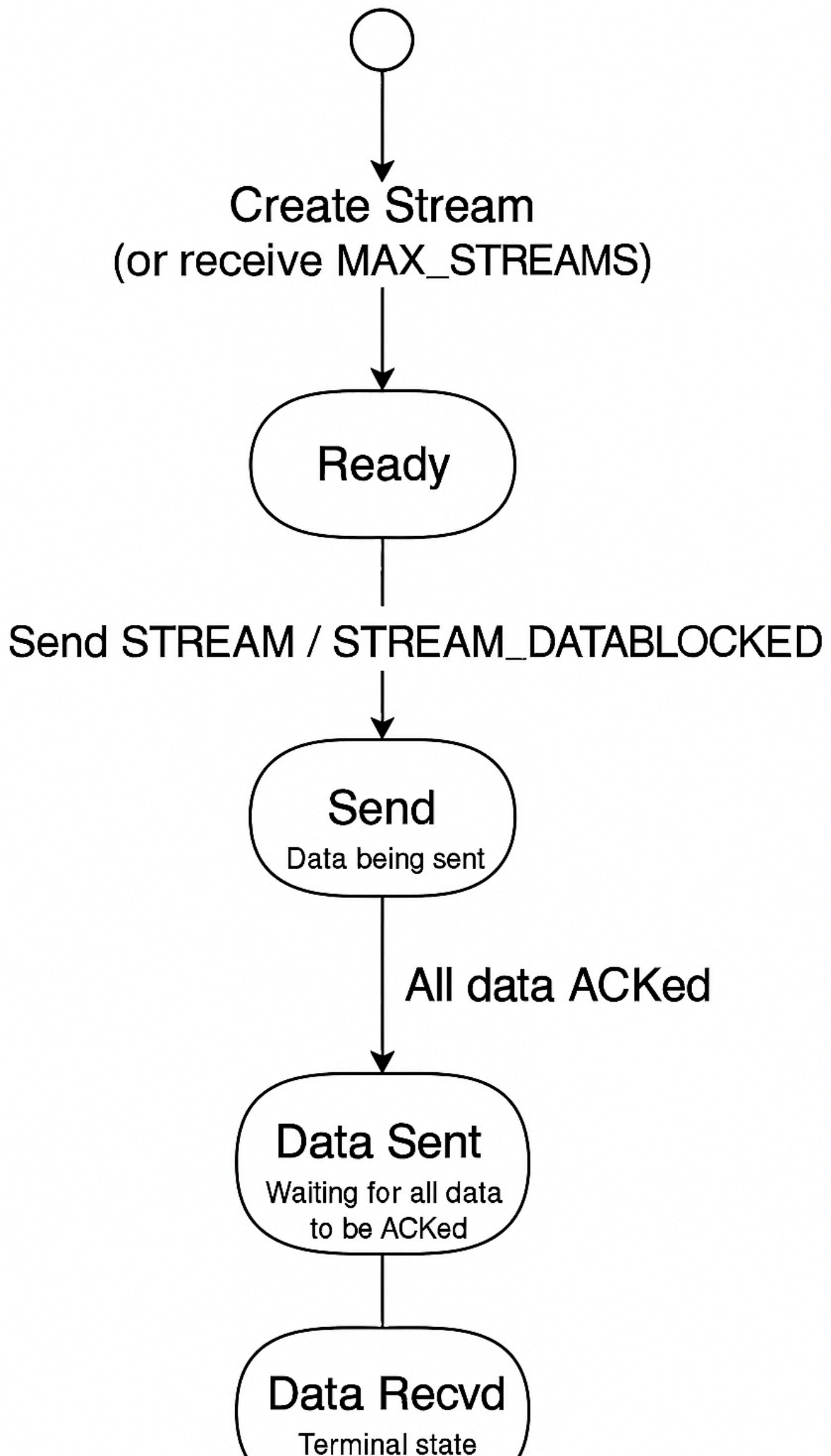
Stream ID Bits 0-1	Type	Initiator
0x00	Client-Initiated, Bidirectional	Client
0x01	Server-Initiated, Bidirectional	Server
0x02	Client-Initiated, Unidirectional	Client
0x03	Server-Initiated, Unidirectional	Server

Stream IDs are 62-bit integers. The two least significant bits encode the type, and the remaining bits increase monotonically. So client-initiated bidirectional streams are 0, 4, 8, 12, ... and server-initiated bidirectional streams are 1, 5, 9, 13, ...

Why streams eliminate HOL blocking: Each stream is independently ordered and delivered. If a packet carrying data for Stream 4 is lost, only Stream 4 stalls waiting for retransmission. Streams 0, 8, 12, etc., continue to deliver data without any delay. This is fundamentally impossible in TCP, where a single lost segment stalls the entire byte stream.

7.2 Stream States

Streams go through well-defined state transitions. For a sending stream:





For a receiving stream, there are analogous states: Recv → Size Known → Data Recvd → Data Read.

A stream can also be abruptly terminated at any point using RESET_STREAM (sender-initiated) or STOP_SENDING (receiver-initiated).

7.3 Flow Control

QUIC implements flow control at two levels:

Connection-level flow control: Limits the total amount of data that can be sent across all streams combined. Controlled by MAX_DATA frames.

Stream-level flow control: Limits the amount of data that can be sent on each individual stream. Controlled by MAX_STREAM_DATA frames.

Additionally, **MAX_STREAMS** frames limit the number of streams that can be created (separately for bidirectional and unidirectional).

When a sender reaches a flow control limit, it sends a DATA_BLOCKED, STREAM_DATA_BLOCKED, or STREAMS_BLOCKED frame to inform the peer. These frames do not increase the limit, They are signals that the sender is being throttled, which can trigger the receiver to increase the limit.

The flow control credit is based on the **absolute byte offset** within the stream, not on the number of bytes in flight. This means that retransmitted data does not consume additional flow control credit.

8. QUIC Encryption Model (RFC 9001)

8.1 Integration with TLS 1.3

QUIC does not use TLS records. Instead, TLS 1.3 handshake messages are carried in **CRYPTO frames** within QUIC packets. The CRYPTO frame provides its own ordering and reliability mechanism (separate from streams), ensuring that TLS messages are delivered in order and retransmitted if lost.

The TLS 1.3 stack in QUIC operates in a stripped-down mode:

- TLS provides the handshake state machine, key derivation, and certificate validation.
- QUIC handles all packet-level protection (encryption and header protection) itself.
- TLS records and the TLS record layer are NOT used. TLS only produces handshake messages and keying material.

8.2 Packet Protection

Each QUIC packet (except Version Negotiation and Retry) is protected using **AEAD (Authenticated Encryption with Associated Data)** encryption. The specific AEAD algorithm depends on the cipher suite negotiated during the TLS handshake. Common choices:

Cipher Suite	AEAD	Hash
TLS_AES_128_GCM_SHA256	AES-128-GCM	SHA-256
TLS_AES_256_GCM_SHA384	AES-256-GCM	SHA-384
TLS_CHACHA20_POLY1305_SHA256	ChaCha20-Poly1305	SHA-256

The AEAD encryption takes:

- **Key:** Derived from TLS key schedule
- **Nonce:** Constructed from the packet number XORed with an IV (also derived from TLS key schedule)
- **Plaintext:** The packet payload (frames)
- **Associated Data (AD):** The unprotected portion of the QUIC header (everything up to and including the packet number)

This means the header is authenticated but not encrypted (it is in the Associated Data), while the payload is both encrypted and authenticated.

8.3 Header Protection

After AEAD encryption, QUIC applies an additional layer called **Header Protection** (RFC 9001, Section 5.4). This encrypts:

- The least significant bits of the first byte (which contain the packet number length and, for Short Headers, the key phase and reserved bits)
- The packet number itself

Header protection uses a **sample** from the encrypted payload as input to a cipher (AES-ECB or ChaCha20, depending on the cipher suite) to generate a mask. This mask is XORed with the protected bits.

Why header protection exists: Without it, the packet number would be in the clear. Packet numbers increase monotonically and would allow observers to trivially detect packet loss, reordering, and estimate sending rates. By encrypting the packet number, QUIC denies observers this information.

The algorithm for removing header protection requires knowing where the packet number is (which depends on the Connection ID length) and having the header protection key. An observer who does not have these keys cannot remove the header protection and therefore cannot determine the packet number or the exact packet number length.

8.4 Key Updates

QUIC supports **key updates** during a connection without renegotiation. Either endpoint can initiate a key update by toggling the **Key Phase bit** in the Short Header. When the recipient sees a flipped Key Phase bit, it knows to derive the next generation of keys using the TLS key update mechanism (HKDF-Expand-Label with a new label).

This provides forward secrecy for post-handshake data: even if a key is compromised, only data encrypted with that specific key generation is exposed. Previous and subsequent key generations remain secure.

The Key Phase bit is itself protected by header protection, so an observer cannot directly see when key updates occur.

8.5 Encryption Levels Summary

To bring this together, here is what is encrypted at each level and what an observer sees:

Timeline of a QUIC connection

Client Initial



Encrypted with
derivable keys
(observer CAN decrypt)
Contains: ClientHello
(with SNI, ALPN, etc)

Server Initial



Encrypted with
Handshake keys
(observer CANNOT decrypt)
Contains: Certificate,
Finished

Client
Handshake



Encrypted with
Handshake keys
(observer CANNOT decrypt)
Contains: Client Finished

1-RTT Data



Encrypted with
Application keys
(observer CANNOT decrypt)
Contains: All application
data, control frames

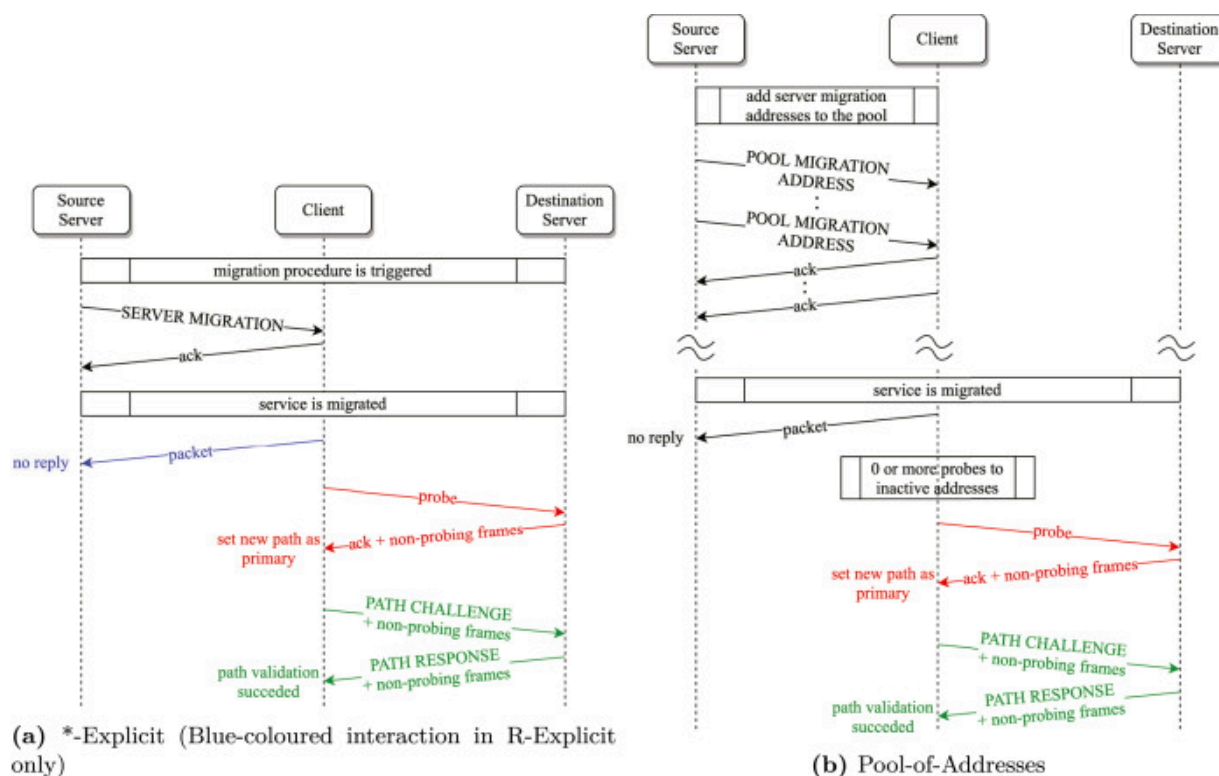
The window where an observer can see meaningful content is limited to the Initial packets, and even that requires active computation (deriving keys from the DCID). After the handshake, everything is opaque.

9. Connection Migration

9.1 What Connection Migration Is

Connection migration is the ability of a QUIC connection to survive changes in the underlying network path, specifically, changes in the IP address or UDP port of either endpoint. When a mobile phone transitions from Wi-Fi to cellular, the phone's IP address changes, but the QUIC connection continues uninterrupted.

9.2 How It Works



1. The endpoint that moves simply starts sending packets from its new address, using the same (or a new) Connection ID.
2. The receiving endpoint detects a packet arriving from an unexpected source address with a valid Connection ID.
3. The receiving endpoint initiates **path validation** by sending a PATH_CHALLENGE frame containing a random 8-byte value.
4. The migrating endpoint responds with a PATH_RESPONSE frame echoing the same 8-byte value from the new address.
5. Once path validation succeeds, the receiving endpoint updates its path state and begins sending to the new address.

Only the client can initiate migration. The server cannot migrate because it cannot prove to the client that a new server address is legitimate (this would open the door to redirection attacks).

9.3 Connection ID Rotation During Migration

When migrating, the client should use a **new Connection ID** (one that was previously provided via `NEW_CONNECTION_ID`) for packets sent from the new address. If the client continues using the same Connection ID from the new address, a network observer who sees both the old and new paths can trivially link them which defeats the privacy benefit of migration.

By using a new Connection ID, the packets on the old path and the new path appear unrelated to a passive observer. The only way to link them would be to observe the CID transition at the server side, which requires being on-path to the server.

9.4 NAT Rebinding vs. Intentional Migration

QUIC also handles **NAT rebinding**, which is when a NAT device assigns a new external port (or IP) to the client's traffic without the client's knowledge. This looks similar to migration (packets arrive from a new source address/port) but is not intentional.

QUIC handles NAT rebinding gracefully: the server performs path validation with the new source address and, upon success, continues the connection. The difference from intentional migration is that the client may not be aware the NAT rebinding occurred.

9.5 Implications for Network Observers

Connection migration fundamentally challenges the notion of a "flow" as defined by a 5-tuple. A single QUIC connection may appear as multiple different 5-tuple flows over its lifetime, each with different Connection IDs. This has significant implications:

- **Firewall state tables** indexed by 5-tuple will see the migrated flow as a new flow.
- **NAT mappings** will need to be re-established.
- **Traffic accounting** based on 5-tuple will fragment a single connection's usage across multiple flow records.
- **Rate-limiting** based on 5-tuple will reset when migration occurs, potentially allowing the connection to temporarily exceed its intended rate limit.

To track QUIC connections across migrations, a network observer would need to correlate Connection IDs across different 5-tuples is made deliberately difficult by CID rotation.

10. Loss Detection and Congestion Control (RFC 9002)

10.1 Loss Detection

QUIC's loss detection is based on acknowledgment processing and is more sophisticated than TCP's. Key mechanisms:

Packet Number Spaces: QUIC maintains three separate packet number spaces: Initial, Handshake, and Application Data. Each space has independent packet numbering starting from 0. Loss detection operates independently in each space.

Packet numbers never repeat. Unlike TCP sequence numbers (which are byte offsets and can wrap), QUIC packet numbers are strictly monotonically increasing. Retransmitted data is sent in a new packet with a new packet number. This eliminates the retransmission ambiguity problem that plagues TCP (where it is sometimes unclear whether an ACK acknowledges the original transmission or the retransmission).

ACK Frames: QUIC ACK frames contain ranges of acknowledged packet numbers (similar to TCP's SACK option, but mandatory and more expressive). An ACK frame can efficiently acknowledge large ranges of packets with gaps.

Loss Declaration: A packet is declared lost if:

1. **Threshold-based:** A sufficient number of later packets have been acknowledged. The default threshold is 3 packets (the "packet threshold"), similar to TCP's fast retransmit.
2. **Time-based:** Enough time has passed since the packet was sent. The time threshold is typically 9/8 of the largest RTT sample (the "time threshold").

Probe Timeout (PTO): If no ACK is received for a period (based on the smoothed RTT plus a margin), QUIC sends a probe packet to elicit an ACK. This replaces TCP's RTO (Retransmission Timeout) mechanism and is designed to be less aggressive in declaring timeout.

10.2 Congestion Control

RFC 9002 specifies a congestion controller based on **NewReno** as the default, but explicitly notes that QUIC is congestion-controller-agnostic. Any congestion control algorithm can be plugged in.

The NewReno-like default:

- **Slow Start:** Congestion window starts small and doubles each RTT.
- **Congestion Avoidance:** After a loss event, the window grows linearly (by approximately one packet per RTT).
- **Recovery:** On loss detection, the congestion window is halved. QUIC enters a "recovery period" and does not reduce the window further for losses during this period.

Why congestion control matters for policing: The congestion controller determines how aggressively QUIC sends data. If a network policer drops QUIC packets to enforce a rate limit, the congestion controller will respond by reducing its sending rate. However, the response dynamics depend on the specific congestion controller being used. Some implementations use BBR (Bottleneck Bandwidth and RTT), which behaves quite differently from NewReno under loss. BBR may not reduce its rate in response to low levels of packet loss, making loss-based policing less effective against BBR-based QUIC flows.

10.3 The Spin Bit

The **Spin Bit** (RFC 9000, Section 17.4) is a single bit in the Short Header that alternates value once per RTT. One endpoint sets it to reflect the value it observed from the peer, creating a "spin signal" that an on-path observer can use to passively measure RTT.

The spin bit is deliberately exposed as one of the few on-path signals, acknowledging that network operators need *some* visibility into connection performance for network management.

However, the spin bit has limitations:

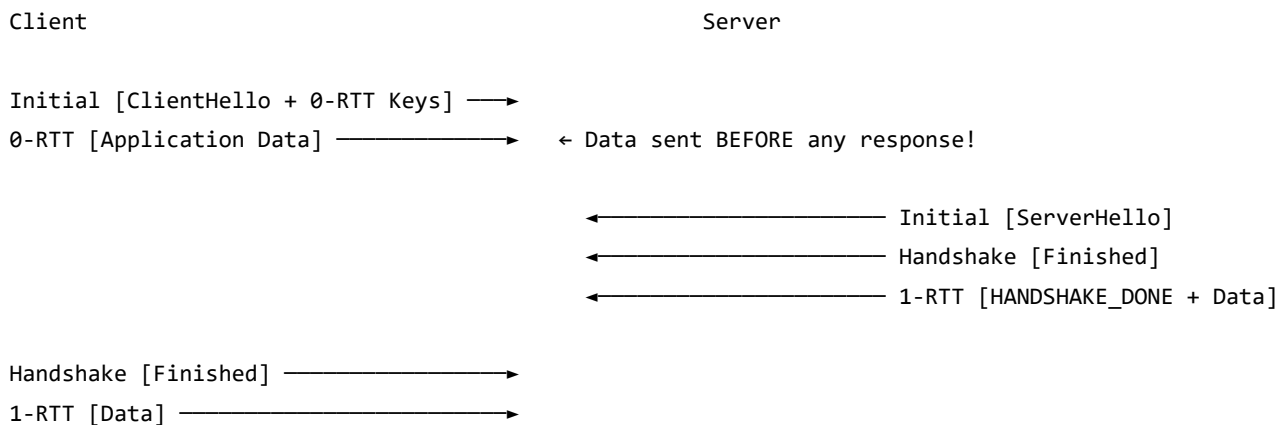
- Implementations may choose to randomly spin it to deny observation (permitted by the spec).
- It only measures RTT, not loss or throughput.

- It is unreliable during low-rate transfers or when packets are reordered.

11. 0-RTT Resumption

11.1 How 0-RTT Works

When a client has previously connected to a server and possesses a valid **Pre-Shared Key (PSK)** from that prior session (delivered via TLS NewSessionTicket), it can send application data in the very first flight alongside the ClientHello, without waiting for any server response.



The 0-RTT data is encrypted using keys derived from the PSK and the ClientHello. The server can process this data immediately after receiving it and verifying the ClientHello.

11.2 Security Risks of 0-RTT

0-RTT data has a fundamental limitation: **it is not protected against replay attacks**. An attacker who captures the client's 0-RTT packets can replay them to the server, causing the server to process the same request multiple times. This is because the 0-RTT data is encrypted with keys that do not include any contribution from the server's nonce (the server has not sent anything yet).

Mitigations:

- Servers should only accept 0-RTT data for **idempotent** operations (e.g., GET requests that do not modify state).
- Servers can maintain a replay cache of previously seen 0-RTT tickets.
- Servers can limit the time window during which 0-RTT is accepted.

11.3 0-RTT on the Wire

From a traffic identification perspective, 0-RTT packets are Long Header packets with type `0x01`. They appear immediately after (or coalesced with) the Initial packet in the first flight from the client. Their presence indicates a resumed connection. If a network observer sees Initial + 0-RTT packets in the same datagram from a client, it knows this is a resumed QUIC session.

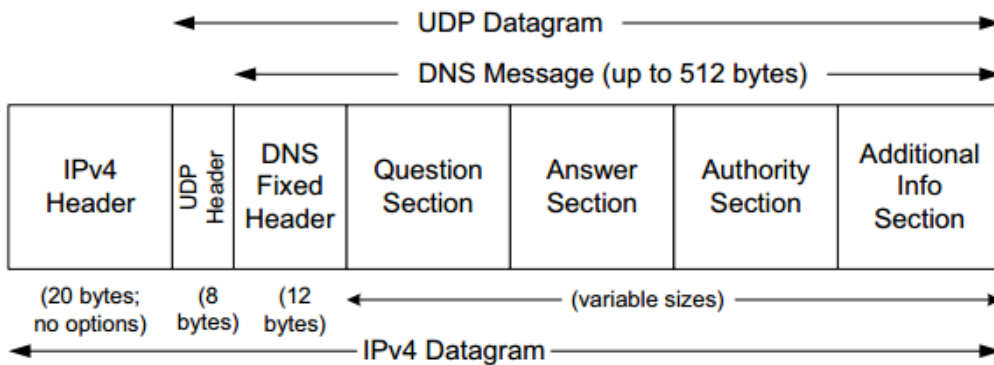
0-RTT packets also have distinct properties:

- They use different encryption keys than Initial packets (derived from the PSK, not derivable by an observer).
- The server may not accept 0-RTT data (it can reject it in its transport parameters).
- If rejected, the client must retransmit the data after the handshake completes using 1-RTT keys.

12. Wire-Level Discrimination — QUIC vs Generic UDP

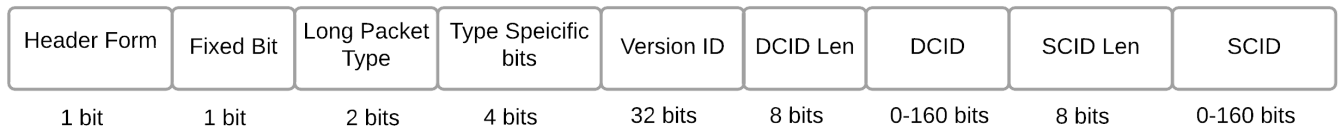
How do you determine if it contains QUIC traffic or some other UDP-based protocol?

DNS UDP :

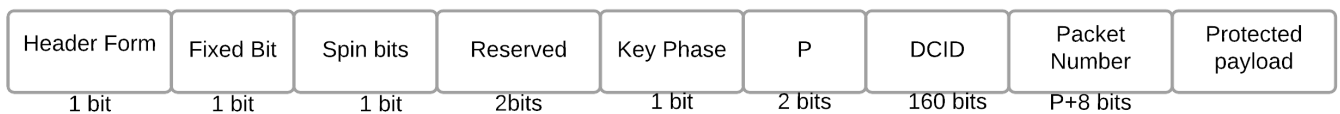


QUIC

Long Header



Short Header



DTLS:

Octet 0		Octet 1		Octet 2		Octet 3	
Version	Traffic Class		Flow Label				
Payload Length				Next Header		Hop Limit	
Source Address (128 bits)							
Destination Address (128 bits)							
Source Port				Destination Port			
Length				Checksum			
Content_type		Version				Epoch	
Epoch		Sequence Number					Length_Record
Length_Record		Message Type		Length_Handshake			
Length_Handshake		Message Sequence				Fragment Offset	
Fragment Offset				Fragment Length			
Fragment Length		Version				Client Random (32 bytes)	
Session_ID Length		Cookie Length		Cipher Suites Length			
Cipher Suites				Comp_method Length		Comp_method	

Generic UDP :

Bit	+0..7		+8..15		+16..23		+24..31
0	Version	Header Length	DSCP	ECN	Total Length		
32	Identification				Flags	Fragment Offset	
64	Time To Live		Protocol		Header Checksum		
96	Source IP Address						
128	Destination IP Address						
160	Options (if present)						
...	Payload						

12.1 The Discrimination Problem

UDP is used by hundreds of protocols: DNS (port 53), DHCP (ports 67-68), NTP (port 123), DTLS, WireGuard, RTP/RTCP (media streaming), various game protocols, SIP, STUN/TURN, custom application protocols, and much more. QUIC is just one of many protocols that run over UDP, and distinguishing it from the rest is non-trivial because:

1. QUIC does not have a dedicated, IANA-assigned port number (though it commonly uses 443).
2. QUIC's payload is encrypted, preventing deep packet inspection of application content.

3. Some other UDP protocols have similar superficial characteristics (e.g., DTLS also encrypts its payload).

12.2 Heuristics for QUIC Identification

Based on the packet format defined in RFC 9000 and the invariants in RFC 8999, the following heuristics can be used:

Heuristic 1: Port Number (Weak Signal)

QUIC typically runs on UDP port 443 (the same as HTTPS). If a UDP datagram's destination port is 443, there is a reasonable probability it is QUIC. However:

- This is not definitive, DTLS also uses port 443.
- QUIC can run on any port. HTTP/3 uses the `Alt-Svc` header or DNS HTTPS record to advertise alternative ports.
- Some QUIC deployments use non-standard ports.

Heuristic 2: First Byte Analysis (Strong Signal)

The first byte of a QUIC packet has a well-defined structure:

- **Long Header:** First bit = 1, second bit = 1 (Fixed Bit). So the first byte has the form `11xxxxxx` (value $\geq 0xC0$). The next two bits encode the packet type.
- **Short Header:** First bit = 0, second bit = 1 (Fixed Bit). So the first byte has the form `01xxxxxx` (value between `0x40` and `0x7F`).

If a UDP datagram starts with a byte in the range `0xC0-0xFF` (for Long Header) or `0x40-0x7F` (for Short Header), and the Fixed Bit is set, it is a strong indicator of QUIC. However, this is not foolproof. Random UDP payloads could coincidentally start with these byte values.

Heuristic 3: Version Field (Very Strong Signal for Long Header)

For Long Header packets, bytes 1-4 contain the QUIC version. If this field matches a known QUIC version (`0x00000001` for v1, `0x6b3343cf` for v2, `0x00000000` for Version Negotiation), the packet is almost certainly QUIC.

The combination of first byte pattern + known version field is a very high-confidence indicator.

Heuristic 4: Connection ID Length Validation (Medium Signal)

In Long Header packets, byte 5 contains the DCID length (0-20), and the byte after the DCID contains the SCID length (0-20). If these values are within the valid range and are consistent with the total packet length, it increases confidence.

Heuristic 5: Minimum Packet Size for Initial Packets

QUIC Initial packets must be padded to at least 1200 bytes (RFC 9000, Section 14.1). So a UDP datagram on port 443 with a Long Header, a valid version field, and a size ≥ 1200 bytes is very likely a QUIC Initial packet.

Heuristic 6: Payload Entropy

QUIC payloads are encrypted and therefore have high entropy (appear random). Generic UDP protocols with plaintext payloads will have lower entropy. However, other encrypted UDP protocols (DTLS, WireGuard) will also have high entropy, so this alone is not sufficient.

Heuristic 7: Temporal Patterns

QUIC connections typically begin with an Initial packet (Long Header) followed by Handshake packets (Long

Header), then transition to Short Header packets. This temporal pattern - Long Headers at the start, Short Headers for the bulk of the connection - is characteristic of QUIC and can be used for flow classification.

12.3 Comparison Table: QUIC vs Other UDP Protocols

Feature	QUIC	DTLS 1.3	DNS (plain)	WireGuard	RTP/RTCP	Generic/Random UDP
Typical port	443	443, various	53	51820	Dynamic (even/odd)	Varies
First byte pattern	0xC0-0xFF or 0x40-0x7F	0x14-0x19 (ContentType) or unified header	Any	0x01-0x04 (msg type)	0x80 (v2) or specific	Arbitrary
Version field at bytes 1-4	Yes (known values)	No (different structure)	No	No	No	No
Payload encrypted	Yes (always)	Yes (after handshake)	No (plaintext)	Yes (always)	Optionally (SRTP)	Usually no
Connection ID present	Yes	Yes (in DTLS 1.3)	No	No (uses index)	SSRC (different semantics)	No
Minimum Initial size	1200 bytes	No such requirement	Usually ≤512 bytes	148-192 bytes	Varies	Varies
Handshake pattern	Long→Short header transition	ContentType change	Single request-response	3 message handshake	Separate signaling	No pattern

12.4 Distinguishing QUIC from DTLS 1.3

DTLS 1.3 is the closest protocol to QUIC on the wire and the most challenging to distinguish. However, there are key differences:

1. **DTLS record format starts with a ContentType byte** (values 20-25 for various record types) or a unified header with different bit patterns. QUIC's first byte has a distinct pattern (fixed bit at position 1).
2. **DTLS does not have a 32-bit version field at bytes 1-4.** DTLS 1.3 has an epoch and sequence number at different offsets.
3. **DTLS does not mandate 1200-byte minimum for its first message.**

The most reliable discriminator is the combination of the first byte pattern and the version field at the correct offset.

12.5 What About QUIC Obfuscation?

There are scenarios where QUIC might be intentionally obfuscated or where other protocols might mimic QUIC patterns:

- **QUIC-like protocols:** Some censorship circumvention tools use protocols that superficially resemble QUIC.
- **QUIC over proxies:** MASQUE (RFC 9298) enables tunneling arbitrary traffic inside QUIC, so the outer flow is QUIC but the inner traffic could be anything.
- **Version Greasing:** QUIC sends Initial packets with greased version numbers (`0x?a?a?a`) to prevent ossification. A detector must not assume only known version numbers are valid QUIC.

12.6 Observable QUIC Flow Characteristics (Summary)

For completeness, here is a consolidated summary of what is visible about a QUIC flow to a passive network observer:

Property	Observable?	Where?
Source/Destination IP	Yes	IP header
Source/Destination Port	Yes	UDP header
UDP payload size	Yes	UDP header
Header form (Long/Short)	Yes	First bit of QUIC header
QUIC version	Yes (Long Header only)	Bytes 1-4
Destination Connection ID	Yes	QUIC header
Source Connection ID	Yes (Long Header only)	QUIC header
Packet type (Initial, Handshake, 0-RTT)	Yes (Long Header only)	First byte, bits 4-5
Spin Bit	Yes (Short Header only)	First byte, bit 2
Packet number	No	Encrypted (header protection)
Frame types	No	Encrypted
Stream data	No	Encrypted
Application protocol (HTTP/3, DNS, etc.)	No	Encrypted (though Initial decryption can reveal ALPN in ClientHello)
SNI (Server Name)	Derivable from Initial packets (by deriving Initial keys and decrypting ClientHello)	Will be hidden by ECH in the future
Number of streams	No	Encrypted
Retransmissions	No (packet numbers are encrypted)	Not directly
RTT	Approximately via Spin Bit	Short Header

Property	Observable?	Where?
Total bytes transferred	Yes	Sum of UDP datagram sizes
Packet inter-arrival times	Yes	Timestamps
Packet size distribution	Yes	UDP datagram sizes
Connection duration	Yes	First to last observed packet

13. QoS and Traffic Policing Mechanisms

This section surveys the established QoS and policing mechanisms that are relevant to network traffic management, independent of QUIC.

13.1 Traffic Policing vs. Traffic Shaping

These are two fundamentally different approaches to rate enforcement:

Traffic Policing:

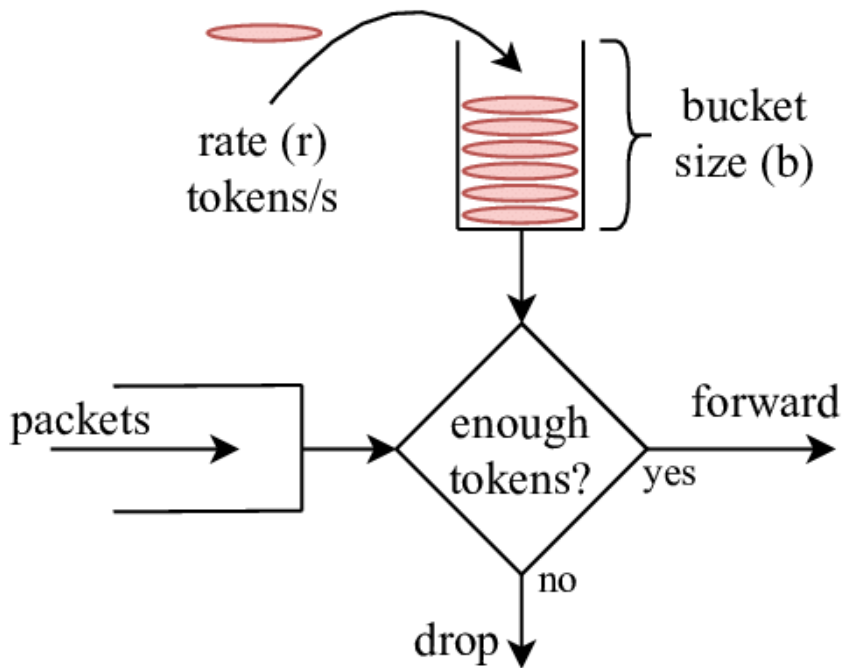
- Operates on a packet-by-packet basis.
- Packets that exceed the rate limit are **dropped** (or re-marked to a lower priority).
- Does not buffer packets. Does not introduce delay.
- Results in a "bursty" loss pattern when the sender exceeds the rate.
- Typically implemented at ingress points (incoming traffic).
- Lower memory requirement (no packet queues needed).

Traffic Shaping:

- Buffers packets and releases them at a controlled rate.
- Packets are **delayed**, not dropped (unless the buffer overflows).
- Smooths out traffic bursts.
- Typically implemented at egress points (outgoing traffic).
- Requires buffer memory proportional to the burst size.
- Introduces latency.

For QUIC traffic management, policing is generally more relevant than shaping because QUIC traffic is typically observed at transit points where buffering is impractical or undesirable.

13.2 Token Bucket Algorithm



The Token Bucket is the most widely used algorithm for traffic policing. It provides a mechanism to enforce an average rate while permitting controlled bursts.

How it works:

1. A "bucket" has a fixed capacity of **B tokens** (the burst size).
2. Tokens are added to the bucket at a constant rate of **r tokens per second** (the committed information rate).
3. When a packet of size **s** arrives:
 - If the bucket contains $\geq s$ tokens: the packet is **conforming**. Remove s tokens from the bucket. Forward the packet.
 - If the bucket contains $< s$ tokens: the packet is **non-conforming**. Either drop it or re-mark it to a lower priority class.
4. Tokens that would cause the bucket to exceed capacity B are discarded (the bucket never overflows).

Key parameters:

- **CIR (Committed Information Rate):** The token generation rate. This is the sustained average rate enforced by the policer.
- **CBS (Committed Burst Size):** The bucket capacity. This determines the maximum burst the policer will tolerate. A larger CBS allows longer bursts above CIR.
- **Relationship:** The maximum burst duration at a rate exceeding CIR depends on both CBS and the excess rate.

Behavioral properties:

- Over long timescales, the average throughput is bounded by CIR.
- Over short timescales, the traffic can burst up to CBS bytes above the sustained rate.
- The token bucket is memoryless in the sense that it only tracks the current token count, not the history of arrivals.

13.3 Single Rate Three Color Marker (srTCM, RFC 2697)

The srTCM extends the basic token bucket to three "colors" using two buckets:

- **Committed Bucket (C):** Capacity = CBS, fills at rate CIR.
- **Excess Bucket (E):** Capacity = EBS (Excess Burst Size), fills from overflow of C bucket.

Packet classification:

1. If C bucket has enough tokens → **Green** (conforming). Deduct from C.
2. Else if E bucket has enough tokens → **Yellow** (marginal). Deduct from E.
3. Else → **Red** (non-conforming).

Green packets are forwarded normally, Yellow packets may be forwarded with lower priority (or dropped under congestion), and Red packets are typically dropped.

13.4 Two Rate Three Color Marker (trTCM, RFC 2698)

The trTCM uses two independently filling buckets:

- **Committed Bucket (C):** Capacity = CBS, fills at rate CIR.
- **Peak Bucket (P):** Capacity = PBS (Peak Burst Size), fills at rate PIR (Peak Information Rate), where $PIR > CIR$.

Packet classification:

1. If P bucket does not have enough tokens → **Red** (exceeds peak rate).
2. Else if C bucket does not have enough tokens → **Yellow** (exceeds committed rate but within peak rate).
3. Else → **Green** (within committed rate).

The trTCM provides finer-grained control by distinguishing between "exceeding the committed rate" and "exceeding the peak rate."

13.5 Leaky Bucket Algorithm

The Leaky Bucket is sometimes confused with the Token Bucket but has a key difference:

- Packets enter a queue (the "bucket").
- Packets are released from the queue at a **constant rate** (the leak rate).
- If the queue is full, incoming packets are dropped.

The Leaky Bucket enforces a **strict constant rate** with no bursting. Every packet exits at the same rate regardless of arrival pattern. This is actually traffic shaping, not policing (since it involves queueing).

Comparison with Token Bucket:

Property	Token Bucket	Leaky Bucket
Bursts allowed?	Yes, up to burst size	No, strictly constant rate
Queueing?	No (policing mode)	Yes (shaping)
Output pattern	Bursty (controlled)	Perfectly smooth

Property	Token Bucket	Leaky Bucket
Memory required	Minimal (just token count)	Proportional to burst size
Latency introduced	None (policing)	Yes (queueing delay)

13.6 Exponentially Weighted Moving Average (EWMA)

EWMA is not a policing mechanism per se, but a **rate estimation technique** often used in conjunction with policing to estimate the current traffic rate.

Formula:

$$\text{rate_estimate}(t) = \alpha \times \text{sample}(t) + (1 - \alpha) \times \text{rate_estimate}(t-1)$$

Where:

- α ($0 < \alpha < 1$) is the **smoothing factor** (weight given to new samples)
- $\text{sample}(t)$ is the current rate measurement (e.g., bytes/second in the last interval)
- $\text{rate_estimate}(t-1)$ is the previous estimate

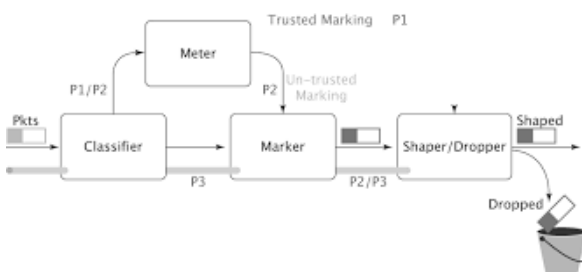
Properties:

- α close to 1: Fast response to changes, but noisy (tracks short-term fluctuations).
- α close to 0: Slow response, but stable (smooths out noise).
- EWMA naturally gives more weight to recent samples and exponentially declining weight to older samples.
- The "memory" of EWMA (the effective number of samples that contribute significantly) is approximately $1/\alpha$.

In the context of traffic policing, EWMA is used for:

1. **Rate estimation:** Computing the smoothed sending rate of a flow for comparison against a threshold.
2. **Anomaly detection:** Detecting sudden rate changes that might indicate a shift in traffic behavior.
3. **Adaptive policing:** Adjusting policing parameters based on the estimated rate trend (e.g., adjusting the token generation rate dynamically).

13.7 Differentiated Services (DiffServ) Architecture



DiffServ (RFC 2475) is the primary QoS framework used in IP networks. It operates on a per-hop basis and classifies packets into a small number of service classes.

Key concepts:

DSCP (Differentiated Services Code Point): A 6-bit field in the IP header (part of the old ToS byte) that indicates the per-hop behavior (PHB) the packet should receive. Routers use the DSCP to determine queuing and dropping

priorities.

Per-Hop Behaviors (PHBs):

PHB	DSCP Value(s)	Service	Description
Default (BE)	000000	Best Effort	Standard forwarding, no special treatment
Expedited Forwarding (EF)	101110	Low latency, low jitter	For real-time traffic (VoIP, video). Gets priority queuing.
Assured Forwarding (AF)	AFxy (4 classes × 3 drop precedences)	Reliable delivery with tiered priority	AF11-AF43. Higher class = higher priority, higher drop precedence = more likely to be dropped under congestion.
Class Selector (CS)	CSx	Backward compatible with IP Precedence	CS0-CS7

In the context of QUIC: The DSCP marking is in the IP header, which QUIC does not control (it is set by the OS or the application). QUIC itself does not interact with DiffServ. However, a network policer can:

1. Read the DSCP of incoming QUIC packets.
2. Re-mark the DSCP based on policing results (e.g., mark conforming packets as AF11 and non-conforming packets as AF13, so they are more likely to be dropped under congestion).

One complication is that QUIC endpoints may set their own DSCP values, and different streams within a QUIC connection might logically deserve different treatment, but since the stream assignment is encrypted, the network cannot differentiate.

13.8 Active Queue Management (AQM)

AQM algorithms manage router queue lengths to avoid buffer bloat and provide congestion signals:

RED (Random Early Detection):

- Drops packets probabilistically as the queue length grows.
- Drop probability increases from 0 (at `min_th` queue length) to a maximum (at `max_th`).
- Above `max_th`, all packets are dropped.
- Provides early congestion signals to senders (via packet loss) before the queue overflows.

CoDel (Controlled Delay):

- Instead of monitoring queue *length*, CoDel monitors queue *delay* (sojourn time).
- If packets consistently experience > 5ms queue delay for > 100ms ("interval"), CoDel starts dropping packets.
- Particularly effective against buffer bloat.

FQ-CoDel (Fair Queuing + CoDel):

- Combines per-flow fair queuing with CoDel AQM.
- Each flow gets its own sub-queue, preventing a single heavy flow from starving others.

- Widely deployed (default in Linux since kernel 3.6).
- The "flow" hash in FQ-CoDel typically uses the 5-tuple (src IP, dst IP, src port, dst port, protocol). QUIC connections using a stable 5-tuple will get consistent treatment. However, if a QUIC connection migrates (changes 5-tuple), it will be hashed into a different sub-queue.

PIE (Proportional Integral controller Enhanced):

- Controls queue delay using a feedback control mechanism.
- Adjusts drop probability based on the deviation of current delay from a target delay.

13.9 Traffic Classification Approaches

Network traffic classification is a prerequisite for applying differentiated QoS. The main approaches are:

1. Port-Based Classification:

- The simplest method: classify traffic based on source/destination port numbers.
- Example: port 443 → HTTPS, port 53 → DNS.
- Increasingly unreliable because many modern protocols use port 443, and applications can use arbitrary ports.
- For QUIC: port 443/UDP is suggestive but not definitive.

2. Deep Packet Inspection (DPI):

- Examines packet payloads to identify protocol signatures and application types.
- Traditional DPI relies on known patterns in unencrypted payloads (e.g., HTTP method strings, DNS query format).
- For QUIC: DPI is severely limited because the payload is encrypted. DPI can only inspect the QUIC header fields that are in the clear (version, connection IDs) and, with effort, the Initial packet's ClientHello (by deriving Initial keys).
- DPI is the traditional workhorse of traffic classification but is being steadily undermined by the move to encrypted protocols.

3. Statistical / Behavioral Classification:

- Classifies flows based on observable traffic characteristics without inspecting payload content.
- Features used include: packet size distribution, inter-arrival times, flow duration, number of packets, bytes/packet ratio, burst patterns.
- Can be effective even against encrypted traffic because encryption does not hide packet sizes or timing patterns.
- Machine learning models (Random Forest, SVM, Neural Networks) are commonly trained on these features.

4. Machine Learning-Based Classification:

- Modern approach using supervised or unsupervised learning on flow-level or packet-level features.
- Common features: statistical summaries of first N packets (sizes, directions, inter-arrival times), flow-level aggregates (total bytes, duration, packet count), payload byte histograms (entropy analysis).
- Models range from traditional ML (Random Forest, Gradient Boosting) to deep learning (1D-CNNs on raw packet bytes, LSTMs on packet sequences, autoencoders for anomaly detection).
- For encrypted traffic like QUIC, ML-based approaches operating on metadata (sizes, timings) rather than content are the most viable classification strategy.

14. Why Traditional QoS Breaks

14.1 The Fundamental Tension

There is a deliberate tension in QUIC's design between **endpoint privacy/security** and **network operator visibility**. QUIC encrypts almost everything beyond the IP and UDP headers. This is a feature, not a bug. It prevents middlebox interference and ossification. But it simultaneously removes the signals that network operators have relied on for decades:

What TCP exposed (and operators relied on):

- SYN/SYN-ACK/ACK flags → Connection setup detection
- FIN/RST flags → Connection teardown detection
- Sequence numbers → Loss estimation, reordering detection
- Window size → Receiver buffer estimation
- ACK numbers → RTT estimation
- Payload (if unencrypted) → Application identification
- TCP options → SACK, timestamps, window scaling

What QUIC hides:

- All of the above. Connection setup, teardown, loss, reordering, flow control, stream activity, application data - all encrypted.

14.2 What Network Operators Can Still See

Despite the encryption, QUIC deliberately or incidentally leaves some signals visible:

1. **The 5-tuple:** Source/destination IP and port, protocol (UDP). Available from IP/UDP headers.
2. **QUIC version (Long Header only):** Identifies the QUIC version and confirms the protocol is QUIC.
3. **Connection IDs:** Available from QUIC headers (with caveats about Short Header CID length).
4. **Spin Bit:** Provides approximate RTT measurement. But implementations can disable or randomize it.
5. **Packet sizes and timing:** The size of each UDP datagram and the timestamps of arrival are visible. These are the primary inputs for statistical classification.
6. **Initial packet content (with effort):** By deriving Initial keys from the DCID, an observer can decrypt the ClientHello and extract SNI, ALPN (Application-Layer Protocol Negotiation, e.g., "h3" for HTTP/3), and supported versions. This requires computational effort and knowledge of the specific QUIC version's key derivation.

14.3 Specific Challenges for QoS and Policing

Challenge 1: Connection Tracking

Without visible SYN/FIN equivalents, detecting when a QUIC connection starts and ends requires heuristics:

- **Start:** Observing a Long Header Initial packet is a strong signal.

- **End:** There is no visible close signal (CONNECTION_CLOSE is encrypted). Operators must use timeout-based expiry — if no packets are seen for some period, assume the connection has ended.

Challenge 2: Flow Identification Across Migration

A migrating QUIC connection changes its 5-tuple and potentially its Connection ID. To an observer, this looks like a completely new flow. Maintaining continuity for QoS purposes across migration is extremely difficult without cooperation from the endpoints.

Challenge 3: Per-Stream QoS

A single QUIC connection may carry multiple streams with different QoS requirements (e.g., a web page with interactive content and background image loading). In TCP/HTTP/2, these would also be multiplexed, but the TCP headers at least allowed connection-level QoS. In QUIC, even connection-level QoS is harder, and per-stream QoS is impossible without endpoint cooperation because stream assignments are encrypted.

Challenge 4: Congestion Control Interactions

When a network policer drops QUIC packets to enforce a rate limit, it relies on QUIC's congestion controller to reduce its sending rate in response. However:

- QUIC's congestion controller is in userspace and can be modified or replaced (BBR vs NewReno vs Cubic vs custom).
- BBR-based QUIC flows may not respond to low levels of loss the way a loss-based controller would.
- An uncooperative QUIC implementation could theoretically ignore congestion signals (though this would violate the protocol specification and harm the network).

Challenge 5: Distinguishing QUIC Sub-Protocols

QUIC carries HTTP/3, DNS-over-QUIC (DoQ), WebTransport, and potentially any protocol built on top of it. The ALPN in the ClientHello indicates the application protocol, but this is only visible in the Initial packet (and only with effort, as it is encrypted with derivable keys). After the handshake, there is no way to distinguish HTTP/3 from DoQ from any other QUIC application by observing packets on the wire.

Challenge 6: ECH (Encrypted Client Hello)

When Encrypted Client Hello is deployed, the SNI in the ClientHello will be encrypted with the server's public key (obtained via DNS). The observer will only see an "outer" SNI pointing to a CDN or fronting domain, not the actual destination. This will remove one of the last remaining classification signals from QUIC's Initial packets.

14.4 The Operator's Remaining Toolkit

Given these constraints, the tools available to network operators for QUIC traffic management are:

1. **Port-based classification:** Identify likely QUIC traffic by port (443/UDP). Imprecise but fast.
2. **First-byte + version heuristic:** High-confidence QUIC detection for Long Header packets.
3. **Initial packet analysis:** Decrypt the ClientHello to extract SNI and ALPN for application-level classification. Limited to the first flight.
4. **Statistical classification:** Use packet size distributions, timing patterns, and flow-level statistics to classify QUIC flows and potentially distinguish application types.
5. **Rate-based policing:** Apply token bucket or EWMA-based rate limits to identified QUIC flows based on their aggregate behavior.
6. **DSCP marking/re-marking:** Mark QUIC packets at the network edge based on classification results, and use DiffServ PHBs for per-hop treatment.

7. **Fair queuing:** Use FQ-CoDel or similar to ensure QUIC flows get fair bandwidth share without needing per-flow identification.

15. Important Considerations

15.1 Privacy Implications of QUIC Identification

Any mechanism that identifies, classifies, or inspects QUIC traffic raises privacy concerns. QUIC's encryption is explicitly designed to protect user privacy. Efforts to circumvent this encryption (e.g., decrypting Initial packets to extract SNI) may conflict with privacy regulations (GDPR, CCPA) or organizational privacy policies. Any traffic identification system must balance operational necessity with privacy obligations.

15.2 The Ossification Risk

QUIC was designed to resist ossification. If middleboxes start relying on specific QUIC header patterns (e.g., specific version numbers, specific CID lengths, specific first-byte values), and a future QUIC version changes these patterns, those middleboxes will break. QUIC uses GREASE to actively combat this. Any QUIC detection or classification system should be designed to handle unknown QUIC versions gracefully, relying only on the invariants specified in RFC 8999.

15.3 QUIC Fallback to TCP

Many QUIC deployments include a fallback mechanism: if QUIC is blocked (some firewalls drop all UDP/443 traffic), the client falls back to TCP+TLS. This means that blocking or excessively policing QUIC does not necessarily reduce the user's access — it may just shift them to a less efficient transport. Aggressive QUIC policing might therefore degrade user experience without achieving traffic reduction.

15.4 Multipath QUIC

Multipath QUIC (draft-ietf-quic-multipath) extends QUIC to use multiple network paths simultaneously. A single logical QUIC connection could send data over both Wi-Fi and cellular interfaces at the same time. This further complicates flow tracking and policing — a single connection spans multiple 5-tuples *simultaneously*, not sequentially.

15.5 QUIC Datagrams (RFC 9221)

QUIC Datagrams extend QUIC with unreliable datagram delivery (alongside its normal reliable streams). Applications like WebRTC, gaming, and real-time media can use QUIC datagrams for latency-sensitive data that does not need retransmission. From a QoS perspective, datagram traffic within a QUIC connection may have very different latency requirements than stream traffic, but this distinction is invisible to the network.

15.6 UDP Amplification and Reflection Attacks

Because QUIC runs over UDP, it is subject to the same amplification attack concerns as any UDP protocol. The 1200-byte minimum Initial packet size and the Retry mechanism (Section 4.3) are QUIC's primary mitigations. However, a policing system must be aware that:

- Spoofed Initial packets can be used to trigger server responses (amplification ratio is bounded by the anti-amplification limit of 3x).
- Version Negotiation and Retry packets are sent in response to Initial packets and could be part of a reflection attack.
- The policing system should consider rate-limiting Initial packets from a single source to mitigate spoofed Initial floods.

15.7 Performance Characteristics of QUIC vs TCP

Aspect	TCP+TLS 1.3	QUIC v1
Connection setup	2 RTT (1 TCP + 1 TLS)	1 RTT (integrated handshake)
Connection resumption	1 RTT (TCP Fast Open + TLS 0-RTT)	0 RTT
HOL blocking	Yes (TCP-level)	No (independent streams)
Connection migration	Not supported	Supported
Userspace implementation	No (kernel TCP)	Yes
CPU overhead	Lower (kernel optimized, hardware offload)	Higher (userspace, no hardware offload yet)
Loss recovery	Per-connection	Per-stream (more granular)

QUIC's userspace implementation means it currently has higher CPU overhead per byte than TCP, particularly because hardware offload (TSO, GRO, checksum offload) does not work for QUIC the way it does for TCP. This is an active area of development (GSO-based batching, io_uring integration, QUIC offload NICs).

15.8 QUIC's Growing Traffic Share

QUIC is not a niche protocol. As of 2026:

- Google Chrome uses QUIC for all Google properties (Search, YouTube, Gmail, Maps, Drive).
- Meta/Facebook uses QUIC extensively for mobile applications.
- Cloudflare, Fastly, and Akamai support HTTP/3 (QUIC) on their CDNs.
- Apple supports QUIC in Safari and iOS.
- Microsoft supports QUIC in Edge and Windows.

Estimates suggest QUIC accounts for 7-9% of global internet traffic and over 30% of web browsing traffic. This share is growing and makes QUIC traffic management an operational necessity, not an academic exercise.